

The Direct Answer

Start with [Pi Agent](#) + [Llama.cpp \(NPU version\)](#).

- **Why Pi Agent?** It is the best "no subscription" starter because it **natively supports local LLMs** (unlike ECC/Superclaude, which often require "hijacking" paid APIs to work locally). It is a lightweight CLI tool that runs perfectly in terminals (like Termux) and doesn't suffer from the "bloat" of larger frameworks. [1]
 - **Why not ECC yet?** "ECC" (Everything Claude Code) is an optimizer for the *Claude Code* workflow. While powerful, it is designed around a paid ecosystem. Using it for free requires complex workarounds (like using "OpenClaw" to mimic the API). Save this for later.
 - **Why not Prime Agent?** "Prime Agent" refers to [ThePrimeagen's '99' agent](#) (the "ponytail caveman" is a reference to his persona). This is fantastic *if and only if* you live inside **Neovim**. If you want a general terminal agent, Pi is better.
-

1. De-Conflicting Your Stack (The "Cancel Out" Question)

Do frameworks "cancel out" your MCP servers?

No. They are opposite sides of a coin.

- **The Frameworks (Pi, Superclaude, ECC)** are the **Boss**. They decide *what* to do.
- **The MCP Servers (Perplexity, Linear, Semgrep)** are the **Employees (Specialists)**. The Boss calls them to get data or do work.
- **Your Plan:** You will configure your chosen **Agent (Pi)** to use your **MCP Servers** as tools. You don't lose them; you finally give them a way to be used.

2. Your "No Subscription" Setup Guide

Since you have the **Qualcomm Genie X / QAIRT SDKs**, you likely have a Snapdragon X Elite laptop or a high-end Android device. This is a goldmine for free AI because you can offload work to the NPU.

Step 1: The Engine (Local Model)

- **Tool:** llama.cpp (NPU version) or Ollama (if easier).
- **Action:** Use your Qualcomm SDKs to run a quantized model (like Llama 3 8B or Mistral) on the NPU. Expose it as a local server (usually `http://localhost:8080`).
- **Why:** This runs the "brain" for free without eating your CPU/Battery.

Step 2: The Interface (The Agent)

- **Tool:** **Pi Agent** (or pi-coding-agent). [2, 3, 4]
- **Action:** Install Pi via npm. Configure it to point to your local llama.cpp server instead of Anthropic/OpenAI.
 - *Tip:* Pi has extensions specifically for llama.cpp and Ollama. [1, 5]
- **Integration:** Point Pi to your **MarkTechpost** repo to "read" the tutorials.

Step 3: The Tools (MCP)

- **Tool:** Your generic MCP servers (Perplexity, Linear, etc.).
- **Action:** specific frameworks like **Superclaude** or **Openship** have native "MCP Client" features. Pi Agent is extensible; check if it has an MCP bridge or use [Open Interpreter](#) (which you already have) as it handles local tools/MCP very well. [4]

3. Categorizing Your Assets (What goes where?)

Use this list to know what to keep installed and what to archive for now.

Category	Asset Name	Status for You
Active Agent (The "Driver")	Pi Agent, Open Interpreter	USE FIRST. These are your main interfaces. Open Interpreter is great for executing code locally.
Niche Agent	Prime Agent (99), Browser-use	Keep. Use '99' only for Neovim. Use 'Browser-use' when you need the agent to click things on a website.
Heavy Frameworks	ECC, Superclaude, Openship, CrewAI	Archive. These are complex orchestrators. Learn the basics with Pi/Open Interpreter first. ECC/Superclaude are often too tied to paid APIs.
Tools / Skills	MCP Servers, Tiamax, Termite	Connect. These are the "skills" your agent calls. Tiamax/Termite are specific to your Termux environment—keep them active.
Optimization	RTK-AI (Rust Token Killer)	USE. Even with local models, this compresses context, making your agent faster and able to "remember" more

Category	Asset Name	Status for You
		history.
Knowledge	MarkTechpost Repo, Unsloth Docs	Reference. Feed these text files into your agent's context (e.g., "Read this doc and tell me how to run X").

Summary: The "Day 1" Action Plan

1. **Ignore:** ECC, Superclaude, and Ringer for now (too complex/cloud-centric).
2. **Install: Pi Agent.**
3. **Run:** A local model using your **Qualcomm NPU SDK** (via llama.cpp).
4. **Connect:** Configure Pi to talk to your local model.
5. **Task:** Ask Pi: *"Read the MarkTechpost tutorials in my folder and summarize the agent memory section for me."* [3, 6]

This uses your **local hardware**, **zero money**, and leverages your **existing repos**.

- [1] <https://www.rushis.com>
- [2] <https://www.youtube.com>
- [3] <https://www.youtube.com>
- [4] <https://github.com>
- [5] <https://pi.dev>
- [6] <https://patloeber.com>